

UART Single-Wire Keyboard Interface Driver Library Technical Specification

Version 1.x

1. Overview

The **BC6xxx series** keyboard interface chips and **BC759x series** LED **7-segment display** driver + keyboard interface chips all provide a unified UART single-wire keyboard interface. This driver library is applicable to all chips using this interface protocol. By using this driver library, users only need to use a very small amount of code (fewer than 10 lines) to implement full keyboard interface functions, including independent key press/release detection, combination keys, and long-press keys.

This driver is written in standard C language and is applicable to all target environments using C. It is also a lightweight; taking the Cortex-M3 as an example, this library occupies only about 400 bytes of program space and 32 bytes of RAM after compilation.

This driver library is highly consistent with the "*BC727x **7-segment display** and Keyboard Interface Chip Driver Library*" in terms of the user interface; when switching to different chips, the user program only needs minimal changes.

Protocol Details

For the UART single-wire keyboard interface, each transmission is a 1-byte value representing a keyboard event.

- **0-0x7F**: Represents the key value for a **press** event.
- **0x80-0xFF**: Represents the key value for a **release** event (where the highest bit acts as a flag).

The theoretical maximum number of keys is 128. The chip with the most keys is currently the BC7591, which supports 96 keys. This driver library utilizes unused key value space (e.g., values not mapped to physical keys) to allow users to define combination keys and long-press keys as custom key values. In the user program, handling combination and long-press keys becomes as simple as handling ordinary keys, as all complex conversions and logics are encapsulated within the driver library.

For example, if the BC7591 physically uses keys 10-96, key values 0-9 can be utilized for user-defined combination or long-press functions, providing a total of 26 available custom key value slots.

Workflow

The library provides an event buffering function, allowing users to query keyboard status when the program is idle. Standard usage involves three steps:

1. Call `update_key_status()` from the UART interrupt.
2. Call `is_key_changed()` in the main loop to query for new keyboard activity.
3. Call `get_key_value()` to retrieve the value and execute response operations.

2. Function Reference Manual

2.1 Setting Functions

`set_detect_mode()`

- **Declaration:** `void set_detect_mode (unsigned char Mode);`
- **Description:** Controls the keyboard detection mode.
 - **Mode 0 (Default):** Only reports key press (conduction) events and ignores releases.
 - **Mode 1:** Detects both presses and releases. `is_key_changed()` will return non-zero twice during a single key cycle.

`set_longpress_count()`

- **Declaration:** `void set_longpress_count (unsigned int CountLimit);`
- **Description:** Sets the threshold for long-press detection. The library tracks how many times `long_press_tick()` is called while a key is held. If the count exceeds `CountLimit`, a long-press event is reported.
- **Default:** 2000.

`set_callback()`

- **Declaration:** `void set_callback (void (*pCallbackFunc) (unsigned char));`
- **Description:** Sets a callback function that is automatically triggered whenever a new key event occurs. **If a callback is set, `is_key_changed()` will no longer report new keys.**
- **Note:** If `update_key_status()` is called within a UART interrupt, the callback will execute as part of that interrupt, extending its duration.

2.2 Definition Functions

def_combined_key()

- **Declaration:** `void def_combined_key (const unsigned char** PCBKeyList, unsigned char* PCBKeyMap, unsigned char CBKeyCount);`
- **Data Structures:** 1. **Combination Definition Array:** `{count, defined_value, key1, key2, ... keyn}`. `count` is the number of keys (max 7).
2. **List Array:** An array of pointers to the combination arrays.
3. **Mapping Array:** RAM space for state tracking (1 byte per combination).

def_longpress_key()

- **Declaration:** `void def_longpress_key (const unsigned char** PLPKeyList, unsigned char LPKeyCount);`
- **Description:** Similar to combination keys but does not require RAM for state caching.
- **Special Case:** Using an original key value of **255 (0xFF)** allows for "No Keyboard Activity" detection.

2.3 Input and Query Functions

update_key_status()

- **Declaration:** `void update_key_status (unsigned char RxData);`
- **Description:** Passes data received via UART to the library. Usually called in the UART ISR.

long_press_tick()

- **Declaration:** `void long_press_tick(void);`
- **Description:** Increments the internal long-press counter. It should be called at a constant pace (e.g., from a timer interrupt or main loop).

is_key_changed() and get_key_value()

- **is_key_changed():** Returns 0 if no new changes, non-zero if a new event (press, release, combo, or long-press) is available.
- **get_key_value():** Retrieves the value and clears the "changed" flag. The library latches the most recent value until it is read or overwritten by a newer event.

3. Usage Examples

3.1 Minimal Application

Only detects key presses, no combination or long-press keys.

```
#include "key_scan.h"

int main(void) {
    while (1) {
        do_something(); // Main Task
        if (is_key_changed()) { // Query status
            unsigned char Key_Value = get_key_value(); // Get value
            switch (Key_Value) {
                case 3: /* Key 3 Action */ break;
                case 24: /* Key 24 Action */ break;
            }
        }
    }
}

void UART_ISR(void) {
    if (RI) { // UART receive interrupt
        update_key_status(RxData);
    }
}
```

3.2 Detecting Key Release Events

```
#include "key_scan.h"

int main(void) {
    set_detect_mode(1); // Enable release detection
    while (1) {
```

```

        if (is_key_changed()) {
            unsigned char Key_Value = get_key_value();
            switch (Key_Value) {
                case 3:                // Key Press
                    do_key3_press_job();
                    break;
                case 0x83:            // Key Release (0x80 | 3)
                    do_key3_release_job();
                    break;
            }
        }
    }
}

```

3.3 Long-press Key Usage - Using Timer Interrupt

```

#include "key_scan.h"
const unsigned char lp1[] = {5, 120};    // Key 5, assigned long-press key value = 120
const unsigned char lp2[] = {20, 121};   // Key 20, assigned long-press key value = 121
const unsigned char* LPDef[] = {lp1, lp2};

int main(void) {
    set_longpress_count(60);    // 3s (50ms * 60)
    def_longpress_key(LPDef, 2);
    // ... logic same as normal key press ...
}
void TIMER_ISR(void) {        // 50ms interval
    long_press_tick();
}

```

3.4 Combination Key Usage

```

const unsigned char cb1[] = {2, 122, 0, 7};    // key0 & key7 combination assigned key value = 122
const unsigned char cb2[] = {3, 123, 11, 15, 20}; // key11, key15, key20 combination key value = 123
const unsigned char* CBDef[] = {cb1, cb2};
unsigned char CMap[2];

int main(void) {
    def_combined_key(CBDef, CMap, 2);
    // ... handle values 122 and 123 in switch ...
}

```

3.5 Detecting Long Period of No Keyboard Activity

```

#include "key_scan.h"
const unsigned char lp1[] = {5, 120};    // Key 5, assigned long-press key value = 120
const unsigned char lp2[] = {20, 121};   // Key 20, assigned long-press key value = 121
const unsigned char no_key[] = {255, 255}; // 'long press' of No-key defined key value = 255
const unsigned char* LPDef[] = {lp1, lp2, no_key};

int main(void) {
    set_longpress_count(60);
    def_longpress_key(LPDef, 3);
    while(1) {
        if (is_key_changed()) {
            unsigned char Key_Value = get_key_value();
            if (Key_Value == 255) { // "No-key" event
                no_key_time++;
                if (no_key_time >= 10) sleep(); // Sleep after 10x timeout
            } else {
                no_key_time = 0; // Counter reset on any other key event
            }
        }
    }
}

```